Week 3: Train and Explain Two Models

Reading Project: Near-Optimal Real-Time Personalization with Simple Transformers

Goal for the week

Train logistic regression and a one-attention-layer predictor on MovieLens. Teach me how logistic regression works, how both models learn, and what you find.

Budget: 6–8 hours, including roughly one hour writing.

1 Reading

1. Jurafsky and Martin, *Logistic Regression*, §§4.2, 4.4–4.5, 4.7–4.8: sigmoid, loss, gradients, and the multiclass case.
2. PyTorch: optimization loop and `CrossEntropyLoss`. No new base-paper sections.

2 The models we are building

Let H be a nonempty set of previously rated movies, n the training-vocabulary size, and y the next recorded movie. Use the same H for both models.

Logistic regression

Use fixed features $x_j(H) = \mathbf{1}_{j \in H}/|H|$. Learn $W \in \mathbb{R}^{n \times n}$ and $b \in \mathbb{R}^n$:

$$z = Wx(H) + b, \qquad p_j = \frac{\exp(z_j)}{\sum_{\ell=1}^{n} \exp(z_\ell)}, \qquad \mathcal{L} = -\log p_y.$$

This is multinomial logistic regression; connect it to the binary sigmoid model.

Attention model

Port Week 2's attention to PyTorch. Gather only the rows for H and compute

$$E_H = \mathrm{softmax}(\boldsymbol{Q}_H \boldsymbol{K}_H^\top)\boldsymbol{V}_H, \qquad h(H) = \frac{1}{|H|}\sum_{i \in H} (E_H)_i, \qquad z = Rh(H) + c.$$

Learn $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ and a linear output layer $R \in \mathbb{R}^{n \times d_v}$, $c \in \mathbb{R}^n$; use the same softmax probabilities and loss as above. Start with $d_{kq} = d_v = 16$ and unscaled attention.

Connection to Week 2. E_H contains its selected attention rows; $h(H)$ represents the user's history. Pooling and readout adapt the layer for prediction rather than directly training Definition 2's reward map. Unselected attention rows are identical and cannot directly distinguish unseen movies. Both models use sets; add no positional encodings.

3 Questions

Keep answers brief.

Q1. Teach me logistic regression

Assume I know linear algebra but have not studied this model. Explain the features, trainable parameters, sigmoid for two classes, and softmax for many classes. Why is the model called linear when its probabilities are nonlinear? *Up to 70 words and the binary probability formula.*

Features: $x(H)$, the averaged one-hot vector of rated movies. Parameters: weights W and bias b. Scores $z = Wx + b$; softmax turns them into probabilities (two classes: sigmoid),

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(w^\top x + b)}}.$$

It is "linear" because the log-odds $w^\top x + b$ are linear in x; the sigmoid/softmax only squashes scores into probabilities, so decision boundaries stay linear.

Q2. Show me how it learns

For three classes, take logits $z = (0, \log 2, 0)$ and correct class $y = 2$. Calculate the probabilities and loss. Derive $\partial\mathcal{L}/\partial z_j$, then explain how gradients update W and b. Check the calculation using PyTorch. *Equations and up to 50 words.*

$p = \text{softmax}(0, \log 2, 0) = (\frac{1}{4}, \frac{1}{2}, \frac{1}{4})$, $\mathcal{L} = -\log\frac{1}{2} = \log 2 \approx 0.693$. Since $\mathcal{L} = -z_y + \log\sum_\ell e^{z_\ell}$,

$$\frac{\partial\mathcal{L}}{\partial z_j} = p_j - \mathbf{1}[j = y] = (\tfrac{1}{4}, -\tfrac{1}{2}, \tfrac{1}{4}).$$

Chain rule: $\partial\mathcal{L}/\partial W = (p - e_y)x^\top$, $\partial\mathcal{L}/\partial b = p - e_y$; gradient descent subtracts η times these, raising the correct class's score and lowering the rest. PyTorch matched (loss 0.6931, same gradient).

Q3. Explain one training example and the comparison

Show one actual history/target pair. State the split dates and retained train/validation/test counts. Identify the inputs and learned parameters of both models. Explain how you prevent future information entering the history, and what attention changes about the representation. *Up to 70 words; compact notation is fine.*

Example (train): user 259, $H = \{255\}$, $y = 286$ (MovieLens IDs). Splits: train $\le$ 1998-02-09 15:47:51 UTC, val $\le$ 1998-03-27 22:06:54, test after; kept 68,521/14,475/14,335. LR: input $x(H)$, learns W, b. Attention: rows $\boldsymbol{Q}_H, \boldsymbol{K}_H, \boldsymbol{V}_H$, learns $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}, R, c$. H uses only same-user events strictly earlier in time. Attention makes each item's vector depend on the other history items; LR uses a fixed average.

Q4. What did you observe?

Report mean test cross-entropy, top-1 accuracy, Hit@10 (target among the ten highest-scoring movies), and parameter count. Explain the result and one limitation; a transformer win is not required. *Up to 60 words.*

Model	Test loss	Accuracy (%)	Hit@10 (%)	Parameters
Logistic regression	6.142	1.91	12.36	2,475,902
Attention model	6.063	1.19	9.97	102,245

Attention achieved lower test loss (6.063 vs 6.142) with 24× fewer parameters, but logistic regression ranked better (Hit@10 12.4% vs 10.0%; accuracy 1.9% vs 1.2%). Both overfit quickly. Limitation: single seed, little tuning, and rating timestamps are only a proxy for interaction order, so differences of this size may not be robust.

4 Code

Use PyTorch in the same repository; built-in layers, autodiff, and optimizers are allowed.

Task 1: prepare one shared dataset

Use MovieLens 100K rating order as a proxy for interaction order; ignore star values.

- Split globally by time near 70%/15%/15%, keeping timestamp ties together. Build the vocabulary using training events only.
- H contains up to 20 most recent distinct in-vocabulary movies for that user with *strictly earlier* timestamps. Skip empty histories and unknown targets; record excluded counts.
- Share examples and all n candidate movies across models. Earlier validation/test events may enter later histories; keep parameters frozen during evaluation.

Task 2: implement and train both models

Create `logistic_regression.py`, `simple_transformer_torch.py`, and a shared `train.py`. Pass **raw logits** to `CrossEntropyLoss`. Fix a seed; select checkpoints by validation loss and test once. Log optimizer, learning rate, batch size, epochs, and runtime.

Task 3: check that learning is real

Check Q2's loss/gradient and agreement with Week 2's attention outputs. On a small batch, verify falling loss and gradients reaching $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ and the readout. Exclude any padding from attention and pooling.

5 Write-up

Save both checkpoints, $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$, movie-ID mapping, cutoffs, and configuration. Fill Q1–Q4 and the fields below; send the PDF **24 hours before our meeting**.

W1. One specific question (or "none after checking")

Does attention's 16-dimensional pooled bottleneck explain why it wins on loss but loses on Hit@10, and would larger d_v (e.g., 64) close the ranking gap?

W2. Repository/commit, run command, and total hours

Repo: github.com/dwc2425/transformer-personalization, commit `e239a6e`. Run: `cd src; python data.py --udata ../ml-100k/u.data --out data_processed.pt; python train.py --data data_processed.pt --model both --lr 5e-3 --epochs 20 --batch_size 512`. Total hours: `4 hours`.

Meeting preparation

Teach logistic regression without notes for five minutes: features, scores, probabilities, loss, and an update. Then explain attention and the training loop.